Model Context Protocol (MCP) com Node.js

Conectando IAs ao seu banco SQLite

Agenda

1. O problema: IAs isoladas dos nossos dados
2. O que é o MCP?
3. Arquitetura: Host, Client e Server
4. As 3 primitivas: **Tools**, **Resources** e **Prompts**
5. Transportes: `stdio` e HTTP
6. **Prática:** servidor MCP de uma loja com **Sequelize + SQLite**
7. Testando com MCP Inspector, cliente Node.js e Claude
8. Segurança e boas práticas

1. O Problema

Modelos de linguagem (LLMs) como Claude, GPT e Gemini são ótimos para conversar, mas:

- **Não enxergam** seu banco de dados, seus arquivos ou suas APIs.
- Cada ferramenta de IA criava **sua própria forma** de integração (plugins, function calling, extensões...).
- Resultado: para **N** aplicações de IA e **M** sistemas, eram necessárias **N × M** integrações.

2. O que é o MCP?

O **Model Context Protocol** é um **protocolo aberto** (criado pela Anthropic em nov/2024 e hoje adotado por vários fornecedores) que padroniza como aplicações de IA se conectam a fontes de dados e ferramentas.

Analogia: o MCP é a **"porta USB-C" das IAs**.
Você escreve **um** servidor MCP e ele funciona em **qualquer** cliente compatível (Claude Desktop, Claude Code, VS Code, Cursor...).

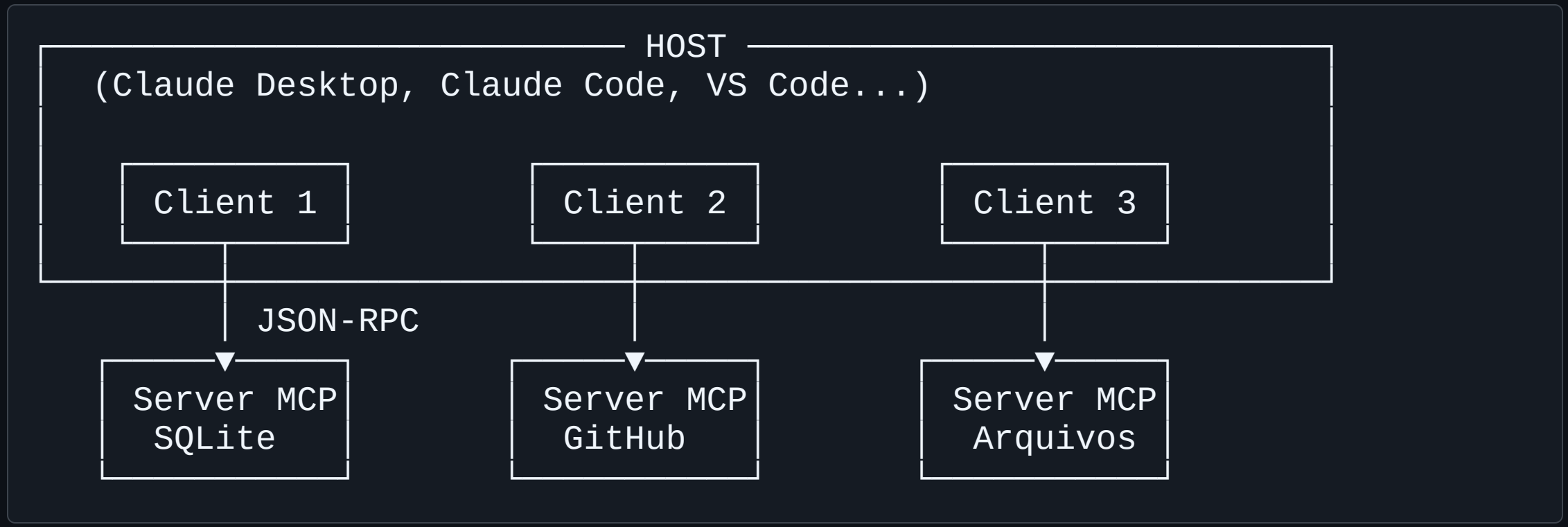
> N × M integrações viram **N + M**.

MCP x API REST

	API REST (Aulas 02 e 03)	MCP
Quem consome	Front-end, apps, outros serviços	**Modelos de IA**
Descoberta	Documentação (Swagger)	**Automática** (`tools/list`)
Formato	HTTP + JSON livre	**JSON-RPC 2.0** padronizado
Descrição	Para humanos	Para a **IA decidir** quando usar

O MCP **não substitui** sua API: muitas vezes um servidor MCP é só uma "camada" que chama sua API ou seu banco.

3. Arquitetura



- **Host:** a aplicação de IA que o usuário usa.
- **Client:** mantém a conexão 1:1 com um servidor.
- **Server: o que nós vamos programar!** Expõe dados e ações.

Como é a conversa (JSON-RPC 2.0)

O cliente pergunta quais ferramentas existem:

```
{ "jsonrpc": "2.0", "id": 1, "method": "tools/list" }
```

A IA decide usar uma ferramenta, e o cliente chama:

```
{
  "jsonrpc": "2.0", "id": 2, "method": "tools/call",
  "params": { "name": "listar_produtos", "arguments": { "categoria": "áudio" } }
}
```

O SDK cuida de tudo isso para nós. 🎉

4. As 3 Primitivas do MCP

Primitiva	Quem controla	Para quê	Analogia REST
Tools	O **modelo** (IA)	Executar ações	`POST`, `PUT`, `DELETE`
Resources	A **aplicação**	Fornecer dados/contexto	`GET`
Prompts	O **usuário**	Modelos de mensagens prontos	"atalhos"

Tools (Ferramentas)

- Funções que a IA pode **chamar** sozinha.
- Têm **nome**, **descrição** e **schema de entrada** (validado com **Zod**).
- A **descrição** é o que a IA lê para decidir quando usar → escreva bem!

Exemplos: `cadastrar_produto`, `atualizar_estoque`, `enviar_email`.

Resources e Prompts

Resources (Recursos)

- Dados identificados por uma **URI**: `sqlite://schema`, `produtos://1`.
- Somente leitura, o host decide quando anexá-los ao contexto.

Prompts

- Templates de mensagem reutilizáveis, com argumentos.
- Aparecem como "comandos" para o usuário (ex.: `/relatorio_estoque`).

5. Transportes

Como as mensagens trafegam entre cliente e servidor:

- `stdio` (entrada/saída padrão)
  - O host **executa** o servidor como um processo local.
  - Ideal para uso local (seu banco SQLite, seus arquivos).
- **Streamable HTTP**
  - O servidor roda como um serviço web (pode usar Express!).
  - Ideal para servidores remotos com vários usuários.

Nesta aula usaremos `stdio`.

⚠️ Regra de ouro do `stdio`

No transporte `stdio`, o **`stdout` é o canal do protocolo**.

```
console.log('debug');   // ❌ QUEBRA a comunicação!
console.error('debug'); // ✅ vai para o stderr, seguro
```

Qualquer texto extra no `stdout` corrompe as mensagens JSON-RPC.

6. Prática: Servidor MCP da Loja

Vamos criar um servidor MCP que permite à IA **consultar e gerenciar produtos** de uma loja guardados em **SQLite**, usando o **Sequelize** (ORM visto na Aula 05).

Tecnologias:

- `@modelcontextprotocol/sdk` → SDK oficial do MCP para TypeScript/JavaScript
- `zod` → validação dos parâmetros das tools
- `sequelize` + `sqlite3` → ORM e driver do SQLite

Passo 1: Configurando o projeto

```
mkdir mcp-loja-sqlite
cd mcp-loja-sqlite
npm init -y
npm pkg set type=module
npm install @modelcontextprotocol/sdk zod sequelize sqlite3
```

- `type=module` → o SDK usa **ES Modules** (`import` / `export`).

Estrutura final:

```
mcp-loja-sqlite/
├── models/
│   └── produto.js   # model Produto
├── sequelize.js     # conexão com o SQLite
├── database.js      # sync + dados iniciais
├── server.js        # servidor MCP
├── client.js        # cliente de teste
└── package.json
```

Relembrando: por que um ORM?

Na **Aula 05** vimos o `sqlite3` puro e depois o **Sequelize**:

```
// SQL puro
db.all('SELECT * FROM produtos WHERE categoria = ?', [cat], (err, rows) => { /* ... */ });

// Sequelize
const rows = await Produto.findAll({ where: { categoria: cat } });
```

- Trabalhamos com **objetos JavaScript** em vez de SQL.
- Métodos `async` → combinam perfeitamente com os handlers do MCP.
- Parâmetros sempre **escapados** → proteção contra SQL Injection.

Passo 2: `sequelize.js` (conexão)

```
import path from 'node:path';
import { Sequelize } from 'sequelize';

const sequelize = new Sequelize({
  dialect: 'sqlite',
  storage: path.join(import.meta.dirname, 'loja.db'),
  logging: false, // IMPORTANTE!
});

export default sequelize;
```

- `storage` → arquivo do banco. Com `import.meta.dirname` ele é encontrado **não importa de qual pasta** o host execute o `node`.
- `logging: false` → veja o próximo slide. 👇

⚠️ Sequelize + `stdio`: cuidado com o log!

Por padrão o Sequelize imprime cada SQL com `console.log`:

```
Executing (default): SELECT `id`, `nome` ... FROM `produtos`;
```

No transporte `stdio` isso vai para o **stdout** e **quebra o protocolo MCP**.

```
logging: false                    // ✅ desliga o log
logging: (sql) => console.error(sql)  // ✅ ou manda para o stderr
```

Passo 3: `models/produto.js`

```
import { DataTypes } from 'sequelize';
import sequelize from '../sequelize.js';

const Produto = sequelize.define('Produto', {
  nome:      { type: DataTypes.STRING,  allowNull: false },
  preco:     { type: DataTypes.FLOAT,   allowNull: false },
  estoque:   { type: DataTypes.INTEGER, allowNull: false, defaultValue: 0 },
  categoria: { type: DataTypes.STRING },
}, {
  tableName: 'produtos',
});

export default Produto;
```

O `id`, `createdAt` e `updatedAt` são criados **automaticamente**.

Passo 4: `database.js` (tabela + dados iniciais)

```
import sequelize from './sequelize.js';
import Produto from './models/produto.js';

export async function iniciarBanco() {
  await sequelize.sync(); // cria a tabela se não existir

  if ((await Produto.count()) === 0) {
    await Produto.bulkCreate([
      { nome: 'Teclado Mecânico', preco: 299.9, estoque: 15, categoria: 'periféricos' },
      { nome: 'Mouse Gamer',      preco: 149.9, estoque: 30, categoria: 'periféricos' },
      { nome: 'Monitor 24"',      preco: 899.0, estoque: 8,  categoria: 'monitores' },
      { nome: 'Headset USB',      preco: 199.5, estoque: 0,  categoria: 'áudio' },
    ]);
  }
}
```

- `sync()` → gera o `CREATE TABLE` a partir do model.
- `bulkCreate()` → insere vários registros de uma vez.

Passo 5: Criando o servidor (`server.js`)

```
import { McpServer, ResourceTemplate } from '@modelcontextprotocol/sdk/server/mcp.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { z } from 'zod';
import sequelize from './sequelize.js';
import Produto from './models/produto.js';
import { iniciarBanco } from './database.js';

await iniciarBanco();

const server = new McpServer({ name: 'loja-sqlite', version: '1.0.0' });

// funções auxiliares para montar as respostas
const texto = (t) => ({ content: [{ type: 'text', text: t }] });
const erro  = (t) => ({ isError: true, content: [{ type: 'text', text: t }] });

// ... tools, resources e prompts aqui ...

const transport = new StdioServerTransport();
await server.connect(transport);
console.error('Servidor MCP loja-sqlite rodando via stdio'); // stderr!
```

Anatomia de uma Tool

```
server.registerTool(
  'nome_da_tool',                    // 1. identificador
  {
    title: 'Título amigável',        // 2. metadados
    description: 'O que faz',        //    (a IA lê isso!)
    inputSchema: { /* zod */ },      // 3. parâmetros validados
  },
  async (args) => {                  // 4. handler
    return {
      content: [{ type: 'text', text: 'resultado' }],
    };
  }
);
```

O retorno é sempre uma lista `content` (texto, imagem, etc.).

Tool 1: Listar produtos (`findAll`)

```
server.registerTool(
  'listar_produtos',
  {
    title: 'Listar produtos',
    description: 'Lista os produtos da loja, opcionalmente filtrando por categoria',
    inputSchema: {
      categoria: z.string().optional().describe('Ex.: periféricos'),
    },
  },
  async ({ categoria }) => {
    const produtos = await Produto.findAll({
      where: categoria ? { categoria } : {},
      raw: true, // devolve objetos simples, prontos para JSON
    });
    return texto(JSON.stringify(produtos, null, 2));
  }
);
```

Tool 2: Cadastrar produto (`create`)

```
server.registerTool(
  'cadastrar_produto',
  {
    title: 'Cadastrar produto',
    description: 'Insere um novo produto na tabela produtos',
    inputSchema: {
      nome: z.string().min(2),
      preco: z.number().positive(),
      estoque: z.number().int().min(0).default(0),
      categoria: z.string().optional(),
    },
  },
  async (dados) => {
    const produto = await Produto.create(dados);
    return texto(`Produto "${produto.nome}" cadastrado com id ${produto.id}`);
  }
);
```

O **Zod** valida a entrada **antes** do handler. O Sequelize faz o `INSERT`.

Tool 3: Atualizar estoque (`findByPk` + `update`)

```
server.registerTool(
  'atualizar_estoque',
  {
    title: 'Atualizar estoque',
    description: 'Soma (ou subtrai, com valor negativo) uma quantidade ao estoque',
    inputSchema: { id: z.number().int(), quantidade: z.number().int() },
  },
  async ({ id, quantidade }) => {
    const produto = await Produto.findByPk(id);
    if (!produto) return erro(`Produto ${id} não encontrado`);

    const anterior = produto.estoque;
    const novo = anterior + quantidade;
    if (novo < 0) return erro(`Estoque insuficiente (atual: ${anterior})`);

    await produto.update({ estoque: novo });
    return texto(`${produto.nome}: estoque ${anterior} -> ${novo}`);
  }
);
```

`isError: true`

- Indica **erro de negócio** para a IA (produto inexistente, estoque insuficiente).
- A IA **lê a mensagem** e pode se corrigir ou explicar ao usuário.
- Erros de **validação do Zod** (ex.: `preco: -5`) são tratados **automaticamente** pelo SDK.

Tool 4: Consulta SQL somente leitura

Às vezes queremos deixar a IA montar o próprio `SELECT`. Criamos uma **segunda conexão**, só-leitura:

```
import path from 'node:path';
import sqlite3 from 'sqlite3';
import { Sequelize, QueryTypes } from 'sequelize';

const sequelizeLeitura = new Sequelize({
  dialect: 'sqlite',
  storage: path.join(import.meta.dirname, 'loja.db'),
  dialectOptions: { mode: sqlite3.OPEN_READONLY }, // 🔒 só leitura
  logging: false,
});
```

Tool 4 (cont.): `sequelize.query()`

```
server.registerTool(
  'consultar_sql',
  {
    title: 'Consultar SQL',
    description: 'Executa uma consulta SELECT no banco da loja. ' +
                 'Use o resource sqlite://schema para ver as tabelas.',
    inputSchema: { sql: z.string().describe('Uma instrução SELECT') },
  },
  async ({ sql }) => {
    if (!/^\s*select\b/i.test(sql)) return erro('Apenas consultas SELECT são permitidas');
    try {
      const linhas = await sequelizeLeitura.query(sql, { type: QueryTypes.SELECT });
      return texto(JSON.stringify(linhas, null, 2));
    } catch (e) {
      return erro(e.message);
    }
  }
);
```

Por que duas camadas de proteção?

1. **Regex** `^select` → recusa logo de cara o que não for consulta.
2. `OPEN_READONLY` → mesmo que algo passe, o **próprio SQLite** recusa escrita:
   `SQLITE_READONLY: attempt to write a readonly database`

```
consultar_sql("SELECT nome, preco FROM produtos WHERE preco > 200")  ✅
consultar_sql("DELETE FROM produtos")  ❌ Apenas consultas SELECT são permitidas
```

> Nunca confie que a IA vai "se comportar". Trate a entrada dela como entrada de **usuário**.

Resource estático: schema do banco

```
server.registerResource(
  'schema',                          // nome
  'sqlite://schema',                 // URI fixa
  { title: 'Schema do banco', description: 'DDL das tabelas', mimeType: 'text/plain' },
  async (uri) => {
    const linhas = await sequelize.query(
      "SELECT sql FROM sqlite_master WHERE type = 'table'",
      { type: QueryTypes.SELECT }
    );
    return {
      contents: [{ uri: uri.href, text: linhas.map((l) => l.sql).join(';\n\n') }],
    };
  }
);
```

Com o schema no contexto, a IA sabe **quais colunas existem** antes de montar um SQL.

Resource dinâmico: produto por id

Usamos um `ResourceTemplate` com parâmetro na URI (parecido com `/produtos/:id` do Express):

```
server.registerResource(
  'produto',
  new ResourceTemplate('produtos://{id}', { list: undefined }),
  { title: 'Produto por id', mimeType: 'application/json' },
  async (uri, { id }) => {
    const produto = await Produto.findByPk(Number(id), { raw: true });
    return {
      contents: [{ uri: uri.href, text: JSON.stringify(produto) }],
    };
  }
);
```

`produtos://1`, `produtos://2`, …

Prompt: relatório de estoque

```
server.registerPrompt(
  'relatorio_estoque',
  {
    title: 'Relatório de estoque',
    description: 'Pede para a IA analisar o estoque',
    argsSchema: { limite: z.string().describe('Estoque mínimo aceitável') },
  },
  ({ limite }) => ({
    messages: [{
      role: 'user',
      content: {
        type: 'text',
        text: `Use a tool listar_produtos e gere um relatório dos produtos
               com estoque abaixo de ${limite}, sugerindo reposição.`,
      },
    }],
  })
);
```

> Argumentos de prompts são sempre **strings**.

7. Testando o Servidor

Três formas:

1. **MCP Inspector** → interface web para testar manualmente
2. **Cliente Node.js** → script que chama o servidor via SDK
3. **Host real** → Claude Desktop / Claude Code / VS Code

Opção 1: MCP Inspector

```
npx @modelcontextprotocol/inspector node server.js
```

- Abre uma página no navegador.
- Abas **Tools**, **Resources** e **Prompts** para listar e executar tudo.
- Mostra as mensagens JSON-RPC trocadas → ótimo para **debug**.

É o "Postman/Insomnia" do MCP.

Opção 2: Cliente Node.js (`client.js`)

```
import { Client } from '@modelcontextprotocol/sdk/client/index.js';
import { StdioClientTransport } from '@modelcontextprotocol/sdk/client/stdio.js';

const transport = new StdioClientTransport({ command: 'node', args: ['server.js'] });
const client = new Client({ name: 'cliente-teste', version: '1.0.0' });
await client.connect(transport);

const { tools } = await client.listTools();
console.log('Tools:', tools.map((t) => t.name));

const r1 = await client.callTool({
  name: 'listar_produtos',
  arguments: { categoria: 'periféricos' },
});
console.log(r1.content[0].text);
```

O cliente **inicia o servidor** como processo filho e conversa via `stdio`.

Opção 2 (cont.): resources e prompts

```
const r2 = await client.callTool({
  name: 'cadastrar_produto',
  arguments: { nome: 'Webcam HD', preco: 249.9, estoque: 5, categoria: 'vídeo' },
});
console.log(r2.content[0].text);   // Produto "Webcam HD" cadastrado com id 5

const r3 = await client.callTool({ name: 'atualizar_estoque', arguments: { id: 4, quantidade: -1 } });
console.log(r3.isError, r3.content[0].text); // true Estoque insuficiente (atual: 0)

const schema = await client.readResource({ uri: 'sqlite://schema' });
console.log(schema.contents[0].text);

const prompt = await client.getPrompt({ name: 'relatorio_estoque', arguments: { limite: '10' } });
console.log(prompt.messages[0].content.text);

await client.close();
```

```
node client.js
```

Opção 3: Usando no Claude Desktop

Edite o arquivo `claude_desktop_config.json`
(Menu **Settings** → **Developer** → **Edit Config**):

```
{
  "mcpServers": {
    "loja-sqlite": {
      "command": "node",
      "args": ["/caminho/absoluto/para/mcp-loja-sqlite/server.js"]
    }
  }
}
```

Reinicie o Claude Desktop e pergunte:

> *"Quais produtos estão sem estoque? Cadastre 10 unidades do Headset USB."*

Opção 3 (cont.): Claude Code e VS Code

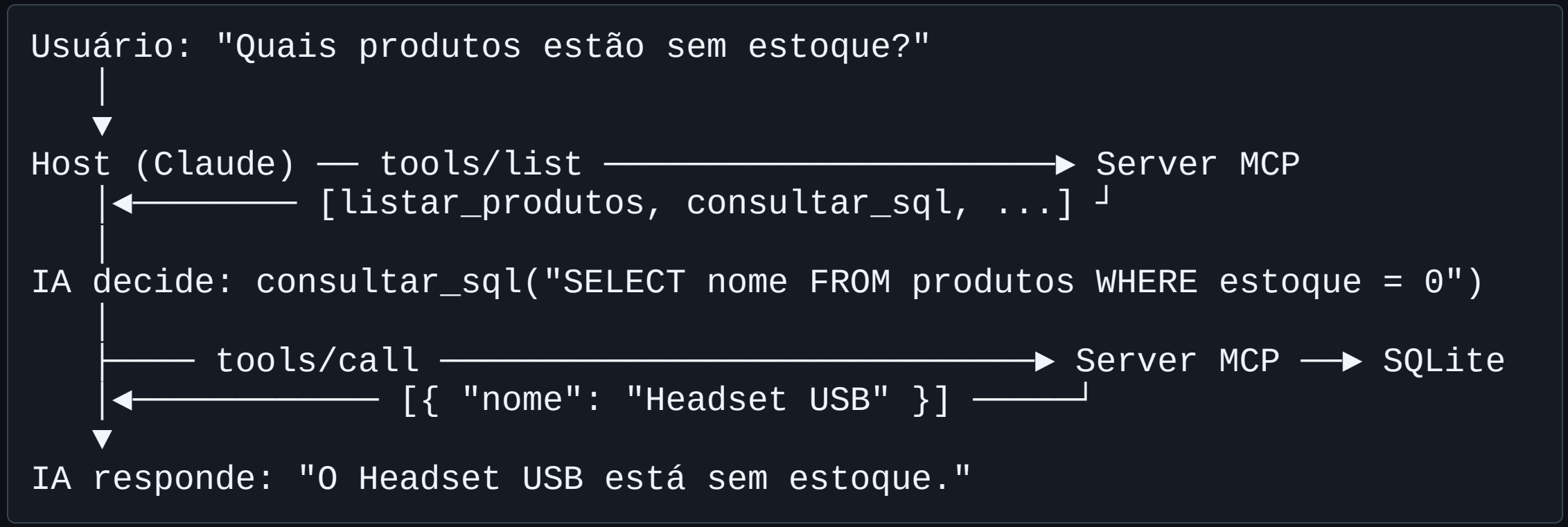
Claude Code (terminal):

```
claude mcp add loja-sqlite -- node /caminho/absoluto/server.js
```

VS Code (`.vscode/mcp.json`):

```
{
  "servers": {
    "loja-sqlite": {
      "type": "stdio",
      "command": "node",
      "args": ["${workspaceFolder}/server.js"]
    }
  }
}
```

O que acontece por trás



8. Segurança e Boas Práticas

- **Valide tudo com Zod:** tipos e limites (`min` , `positive`).
- **Prefira o ORM** (`findAll` , `create`) → sem SQL Injection.
- **Menor privilégio:** `OPEN_READONLY` para consultas livres.
- **Evite expor** `DELETE` **.** Se expor, peça confirmação.
- **Descrições claras:** a IA escolhe as tools pela `description` .
- **Nada no** `stdout` **:** `logging: false` no Sequelize!
- **Segredos em variáveis de ambiente** (`env` na config do host).
- Servidores HTTP precisam de **autenticação** (lembra do JWT?).

Resumo

- **MCP** padroniza a conexão entre IAs e sistemas: "USB-C das IAs".
- **Host** → **Client** → **Server**, conversando via **JSON-RPC 2.0**.
- **Tools** (ações), **Resources** (dados) e **Prompts** (templates).
- Com `@modelcontextprotocol/sdk` + `zod` + **Sequelize/SQLite** criamos um servidor completo em poucas linhas.
- Teste com **MCP Inspector**, um **cliente Node.js** ou um **host real**.
- Segurança: validação com Zod, ORM, conexões só-leitura.

Exercícios

1. Crie a tool `remover_produto` que só remove se o estoque for **0**.
2. Crie o model `Venda` (associado a `Produto` com `belongsTo`) e a tool `registrar_venda`, que insere a venda **e** baixa o estoque dentro de um `sequelize.transaction()`.
3. Crie o resource `categorias://lista` com as categorias distintas.
4. Crie o prompt `sugerir_promocao` que recebe uma `categoria`.
5. **Desafio:** transforme o banco da sua API do Trabalho em um servidor MCP.

Perguntas?

Bora conectar uma IA ao nosso banco!